Higher-Level Sequence Patterns

Lecture 24

CS 1001

Kirk Duran

Today

- Move from reasoning about **loop mechanics** to reasoning about **sequence-level intent**.
- Give formal names to transformation and selection patterns already implemented with loops.
- Refactor suitable loop-based list construction into list comprehensions.
- Trace comprehensions using the same binding, condition, and evaluation model used for `for` loops.
- Distinguish computations that fit a comprehension from those that remain clearer as explicit loops.
- Look briefly at the runtime mechanism that allows `for` to traverse a sequence.

Key idea

Lecture 24 does not introduce a new kind of computation. It introduces a **higher-level representation** for some computations students already know how to express with loops.

The Boundary for This Lecture

Today

- map / transformation as a pattern
- filter / selection as a pattern
- list comprehensions
- semantic equivalence with simple loops
- choosing a readable representation

Not today

- `map()` or `filter()` built-ins
- a catalog of sequence built-ins
- nested comprehensions
- nested data structures
- searching or sorting algorithms

Part 1: From Loop Mechanics to Computational Intent

The Iteration Unit Has Three Levels

Lecture	Primary question	Level
22	How does a `for` loop execute?	mechanism
23	What recurring loop structures solve common tasks?	construction
24	What computation does this loop represent?	abstraction

Key idea

We are moving upward in abstraction, not replacing the execution model from Lecture 22.

From Mechanism to Intent

Mechanistic description

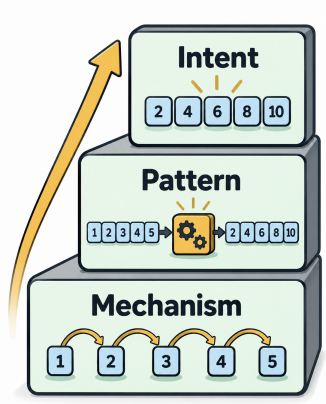
Create an empty list, visit each input value, compute a result, append that result, repeat until the input is exhausted.

Intent-level description

Produce one transformed output value for each input value.

Key idea

Both descriptions are correct. They answer different questions.



Formal Names for Familiar Patterns

Pattern	Relationship	Typical result
transformation / map	each input produces a related output	new sequence
selection / filter	only qualifying inputs remain	new sequence
accumulation	repeated updates summarize the input	one value

Terminology

Map and **filter** name computational patterns here. We are not introducing Python's `map()` or `filter()` functions.

Naming the pattern lets us recognize the same computational relationship even when the particular arithmetic or data changes.

Refactoring Should Preserve Meaning

Definition

A **refactoring** changes how code is expressed while preserving its intended observable behavior.

When we replace a simple list-building loop with a comprehension, we should preserve:

- which input values are examined;
- which conditions are checked;
- which result values are produced;
- the order of the produced values; and
- whether the original input is mutated.

Key idea

Shorter syntax is useful only when it preserves both behavior and clarity.

Part 2: List Comprehensions as Refactored List Construction

The Same Relationship as a List Comprehension

Explicit construction

```
result = []
for value in values:
  result.append(value * 2)
```

Comprehension

```
result = [value * 2
      for value in values]
```

Key idea

The comprehension removes bookkeeping that is predictable from the pattern: creating the empty list and explicitly calling `append`.

A List Comprehension Is an Expression

Definition

A **list comprehension** is an expression that constructs a new list by repeatedly evaluating an output expression over values obtained from an input sequence.

General form

```
[output_expression for name in sequence]
```

Because it is an expression, the completed list can be:

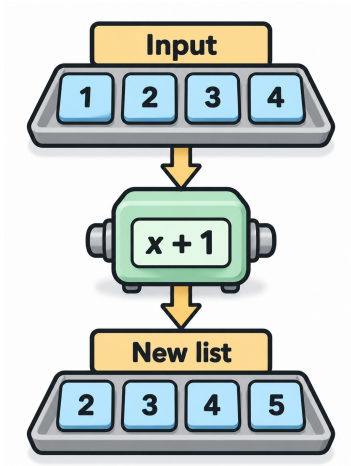
- assigned to a name;
- returned from a function; or
- used as part of another expression.

Anatomy of a Comprehension

Example

```
[x + 1 for x in values]
```

Part	Role
`values`	source sequence
`x`	repeatedly bound name
`x + 1`	output expression



Trace It Using the Old Loop Model

For:

Expression

```
[x + 1 for x in [1, 2, 3, 4]]
```

Step	x	Produced value
1	1	2
2	2	3
3	3	4
4	4	5

Final value: `[2, 3, 4, 5]`

Key idea

The compact syntax does not change the binding model: obtain a value, bind the name, evaluate the output expression, and add the produced value to the new list.

The Output Expression Can Call a Function

Code

```
def square(n:  int) -> int:
   return n * n

squares = [square(x) for x in [1, 2, 3, 4]]
```

The comprehension causes four separate calls:

`square(1)` $\rightarrow$ `1` `square(2)` $\rightarrow$ `4` `square(3)` $\rightarrow$ `9` `square(4)` $\rightarrow$ `16`

The Input Is Not the Output

Input

```
values = [2, 4, 6]
```

Afterwards:

Comprehension

```
doubled = [x * 2 for x in values]
```

`values` $\rightarrow$ `[2, 4, 6]`
`doubled` $\rightarrow$ `[4, 8, 12]`

Key idea

A list comprehension constructs a **new list**. It does not implicitly mutate the source list.

Part 3: Selection and Conditional Production

Selection Has the Same Refactoring Idea

Explicit loop

```
result = []
for x in values:
   if x > 0:
      result.append(x)
```

Filtered comprehension

```
result = [x for x in values
       if x > 0]
```

The condition determines whether a candidate contributes a value to the result.

General Form with a Condition

Filtered comprehension

```
[output_expression for name in sequence if condition]
```

For each candidate:

1. bind the candidate to `name`;
2. evaluate `condition`;
3. if the condition is `True`, evaluate the output expression;
4. otherwise produce nothing for that candidate.

Trace Selection Explicitly

Expression

```
[x for x in [-2, 5, 0, 7] if x > 0]
```

x	x ¿ 0	Produced value
-2	False	—
5	True	5
0	False	—
7	True	7

Final value: `[5, 7]`
Selected values appear in the same relative order in which they were encountered.

Selection and Transformation Can Be Combined

Example

```
[x * 2 for x in values if x > 0]
```

Conceptually:

candidate $\longrightarrow$ **condition** $\longrightarrow$ **transform if selected**

- some candidates produce no output value;
- selected candidates can be transformed before entering the result;
- the result is still a newly constructed list.

Written Order Is Not Evaluation Order

Consider:

Expression

```
[transform(x) for x in values if check(x)]
```

For each `x`, Python conceptually does:

1. evaluate `check(x)`;
2. if it is `True`, evaluate `transform(x)`;
3. place the transformed value into the result.

Common reading mistake

The output expression appears first syntactically, but it is not evaluated for candidates rejected by the condition. If `check(x)` is false, `transform(x)` is not called for that candidate.

A Reliable Mental Expansion

When a comprehension feels dense, expand it mentally.

Comprehension

```
result = [f(x) for x in values if p(x)]
```

Equivalent reasoning structure

```
result = []
for x in values:
   if p(x):
      result.append(f(x))
```

Key idea

The explicit loop remains the tracing model even when the program uses compact syntax.

Part 4: How Do Loops Work “Under the Engine”?

The Surface Syntax Hides a Runtime Conversation

When Python executes:

Surface syntax

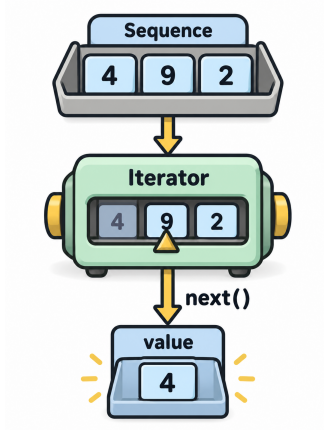
```
for value in sequence:
    use(value)
```

it needs a runtime mechanism that can answer two questions repeatedly:

1. Is there another value?
2. If so, what is it?

Flavor, not required syntax

You are not expected to write the lower-level mechanism yourself in this course.



Conceptual Runtime Model

At a high level, Python behaves approximately like this:

Conceptual model

```
iterator = iter(sequence)
while another value is available:
   value = next(iterator)
   execute loop body
```

Internally, the iterator maintains enough state to know which value should come next. When no next value exists, the runtime reports `StopIteration`; the `for` statement handles that termination automatically.

Key idea

The `for` statement is a high-level control structure built on a lower-level mechanism for requesting successive values.

Why `range` Works Naturally with `for`

A `range` value represents a numerical progression using a start, stop, and step.

Example

```
for i in range(2, 8, 2):
```

The loop receives:

$$2 \to 4 \to 6$$

The range value can support iteration without first constructing a list containing every integer in the progression.

Comprehensions Use the Same Iteration Machinery

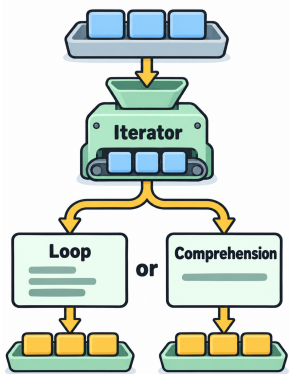
A comprehension does not remove iteration from the computation.

Source

```
[x * 2 for x in values if x > 0]
```

Conceptually, Python still:

- obtains successive source values;
- performs repeated bindings;
- checks the condition;
- evaluates the output expression;
- constructs a new list.



The abstraction hides predictable bookkeeping while preserving the same underlying traversal.

Part 5: Choosing the Right Representation

A Comprehension Has a Narrow Job

A comprehension is usually a strong fit when all of the following are true:

- the goal is to construct one new list;
- values come from one straightforward source sequence;
- each selected candidate produces at most one output value;
- the transformation is short enough to understand locally;
- the condition, if present, is similarly simple.

Good fit

```
[word.lower() for word in words if word != ""]
```

An Explicit Loop Is Better for Richer State Changes

Prefer a normal loop when the body coordinates several pieces of state or several actions.

- multiple accumulators;
- several independent updates;
- complex conditional branching;
- intentional mutation of existing data;
- multiple side effects;
- intermediate states that matter for debugging or explanation.

Key idea

A comprehension is not an "advanced loop." It is a specialized expression for constructing a list.

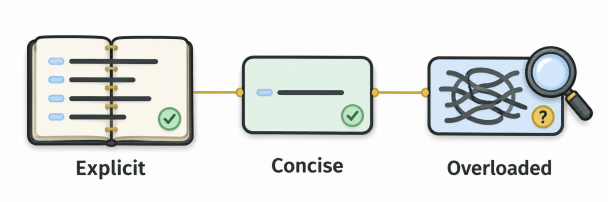
Shorter Is Not Automatically Clearer

Compact and clear

```
[x * 2 for x in values if x > 0]
```

Too much hidden in one expression

several conditions + several calls + several transformations



Key idea

Choose the representation that makes the computation easiest to verify, explain, and maintain.

Debugging Can Favor the Explicit Form

Suppose a comprehension produces the wrong list.
One useful debugging move is to temporarily expand it:

Compact version

```
result = [f(x) for x in values if p(x)]
```

Debugging version

```
result = []
for x in values:
   if p(x):
      result.append(f(x))
```

The expanded form exposes intermediate state and gives the debugger more obvious stopping points.

Semantic Equivalence Checklist

Before replacing a loop with a comprehension, verify:

1. Is the same source sequence traversed?
2. Are candidates examined in the same order?
3. Is the same condition applied to each candidate?
4. Is the same transformation applied to selected candidates?
5. Is a fresh result list produced?
6. Is the original input left unchanged when that is the intended contract?

Part 6: Synthesis

One Problem, Three Levels of Description

Consider producing the lengths of every word.

Level	Description
execution	repeatedly bind a word and execute statements
loop pattern	build a new list by transforming each input
sequence pattern	map each word to its length

Comprehension representation

```
[len(word) for word in words]
```

Map, Filter, and Accumulate Are Different Relationships

Pattern	Input-to-output relationship	Natural representation
map	one output per input	loop or comprehension
filter	zero or one output per input	loop or comprehension
accumulate	many inputs update shared summary state	explicit loop

Key idea

Not every repeated computation should become a comprehension.

Integrated Prediction

Do not treat this as one opaque expression.

Code

```
def adjust(n:  int) -> int:
   return n * 3

values = [-2, 0, 1, 4]
result = [adjust(x) for x in values if x > 0]
```

Predict:

1. every value bound to `x`;
2. every result of `x > 0`;
3. exactly which calls to `adjust` occur;
4. the final value of `result`.

Integrated Trace

x	x ¿ 0	adjust(x)
-2	False	not called
0	False	not called
1	True	3
4	True	12

Therefore:

```
result = [3, 12]
```

The compressed syntax still obeys the same rules for bindings, conditions, function calls, and returned values.

Iteration Unit: The Complete Picture

Lecture	Core responsibility
22	execution mechanics of `for` and `range`
23	constructing useful computations with recurring loop patterns
24	recognizing intent and selecting a higher-level representation

Key idea

The strongest mental model works at multiple levels: understand the mechanism, recognize the pattern, then choose the clearest representation.

Takeaways

1. Map and filter name relationships between an input sequence and a result sequence.
2. A list comprehension is an expression for constructing a new list from those relationships.
3. Comprehensions still rely on repeated binding and iteration; they hide bookkeeping rather than eliminate computation.
4. Conditions are evaluated before the output expression for a candidate is produced.
5. Explicit loops remain preferable when the computation manages richer state or several actions.
6. Understanding the lower-level iteration mechanism explains why the same `for` syntax works across different iterable values.

Next

We leave sequential data as the primary structure and introduce **dictionaries and associative data**.